

Context free Grammar

Definition of context free grammar (CFG):

It is a formal grammar, which consists of a set of production rules. These production rules are used to generate the string of a language.

CFG 'G' can be defined by a 4 tuples as

$$G = (V, T, P, S)$$

$T \rightarrow$ set of terminal symbols (lowercase)

$V \rightarrow$ set of non terminal symbols (uppercase)

$S \rightarrow$ start symbol (from V)

$P \rightarrow$ set of production rules which are used to replace non-terminal symbols in a string with other terminal or non terminal symbols.

Problems:

Construct CFG for the language having any number of a's

$$L = \{\epsilon, a, aa, aaa, aaaa, \dots\}$$



$$G = (V, T, P, S)$$

$$\therefore V = \{S\}$$

$$T = \{a\}$$

$$\delta(S, a) = S$$

production rule $P = \begin{cases} S \rightarrow aS & \text{Rule 1} \\ S \rightarrow \epsilon & \text{Rule 2} \end{cases}$

S is start symbol.

Let us derive one example string "aaaaa" $\in L$ using production rule.

Begin with start symbol S

$\Rightarrow aS$ Rule 1

$\Rightarrow aaS$ Rule 1

$\Rightarrow aaas$ Rule 1

$= aaaaa$ Rule 2 Replace S by ϵ

$= \underline{\text{'aaaaa'}}$ string derived finally

So grammar is correct.

② Construct CFG for the language having any number a's & b's

$L = \{ \epsilon, a, b, aa, ab, bb, ba, bbb, \dots \}$

$P = \{ S \rightarrow aS \mid bS \mid \epsilon S \}$ or $S \rightarrow aS$

$S \rightarrow bS$

$S \rightarrow \epsilon$

$\rightarrow \textcircled{S} a, b$

CFG: $G = (V, T, P, S)$

$G = (\{S\}, \{a, b\}, [S \rightarrow aS \mid bS \mid \epsilon], S)$

③ Design CFG for the language having even number of a's



$G = (V, T, P, S)$

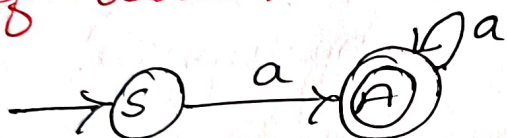
$V = \{S, A\}$

$T = \{a\}$

$$P = \{ S \rightarrow aA \mid \epsilon \\ A \rightarrow as \}$$

S is start symbol

- (4) obtain a grammar to generate string consisting of atleast 1a



$$G = (V, T, P, S)$$

$$V = \{S, A\}$$

$$T = \{a\}$$

$$P = \{ S \rightarrow aA \\ A \rightarrow aA \mid \epsilon \}$$

S is start symbol.

- (5) obtain a grammar to generate string consisting of multiple of 3 a's



$$S \rightarrow aaas \mid \epsilon$$

or



$$G = (V, T, P, S)$$

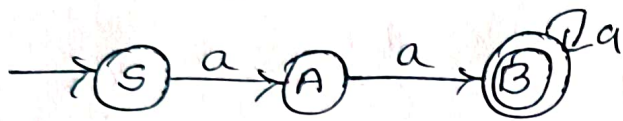
$$V = \{S, A, B\}$$

$$T = \{a, b\}$$

$$P = \{ S \rightarrow aA \\ A \rightarrow aB \\ B \rightarrow as \\ S \rightarrow \epsilon \}$$

S is start symbol

- ⑥ obtain grammar to generate string consisting of atleast two a's.



$$G = (V, T, P, S)$$

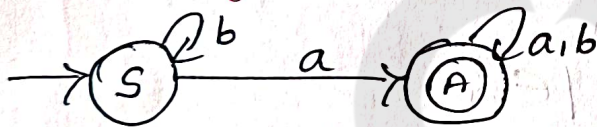
$$V = \{S, A, B\}$$

$$T = \{a\}$$

$$P = \{ S \rightarrow aA \\ A \rightarrow aB \\ B \rightarrow aB | \epsilon \}$$

S is start symbol.

- ⑦ obtain grammar to generate string consisting of a's & b's atleast one a.



$$G = (V, T, P, S)$$

$$V = \{S, A\}$$

$$T = \{a, b\}$$

$$P = \{ S \rightarrow bS \\ S \rightarrow aA \\ A \rightarrow \epsilon | aA | bA \}$$

S is start symbol

- ⑧ obtain grammar to generate string of a's & b's such that string length is multiple of 3

$$S \rightarrow \epsilon | AAA S \\ A \rightarrow a | b$$

$$G = (V, T, P, S)$$

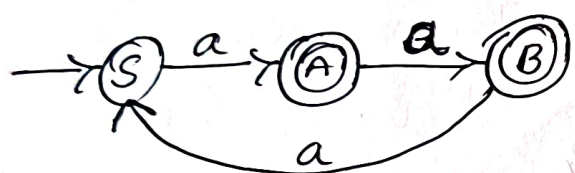
$$V = \{S, A\}$$

(3)

$$P = \{ S \rightarrow \epsilon \mid AAA S \\ A \rightarrow a \mid b \}$$

S is start symbol.

obtain grammar to accept the following language $L = \{ w : |w| \bmod 3 > 0 \text{ where } w \in \{a, b\}^*$



$$\text{or } S \rightarrow a \mid aa \mid aaa S$$

$$G = (V, T, P, S)$$

$$V = \{ S, A, B \}$$

$$T = \{ a \}$$

$$P = \{ S \rightarrow aA \\ A \rightarrow aB \mid \epsilon$$

$$B \rightarrow aS \mid \epsilon \}$$

S is start symbol

obtain a grammar to generate the following language.

$$L = \{ a^n b^n \mid n \geq 0 \}$$

$$L = \{ ab, aabb, aaabbb, \dots \}$$

$$G = (V, T, P, S)$$

$$V = \{ S \}$$

$$T = \{ a, b \}$$

$$P = \{ S \rightarrow \epsilon \mid asb \}$$

S is start symbol.

⑪ obtain a grammar to generate the following language.

$$L = \{a^n, b^n : n \geq 1\}$$

$$L = \{ab, aabb, aaabbb, \dots\}$$

$$G = (V, T, P, S)$$

$$V = \{S\}$$

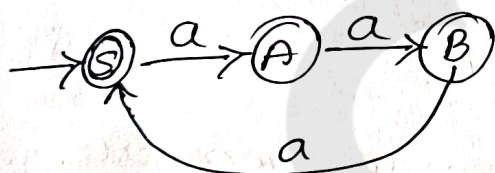
$$T = \{a, b\}$$

$$P = \{S \rightarrow ab | asb\}$$

S is start symbol.

⑫ obtain a grammar to generate the following language

$$L = \{w : |w| \bmod 3 = 0, \text{ where } w \in \{a\}^*\}$$



$$G = (V, T, P, S)$$

$$V = \{S, A, B\}$$

$$T = \{a\}$$

$$P = \{S \rightarrow aA | \epsilon, A \rightarrow aB, B \rightarrow aS\}$$

S is start symbol

⑬ obtain a grammar to generate the following language

$$L = \{a^{n+1}, b^n : n \geq 0\}$$

$$L = \{a\epsilon, aab, aaabb, aaaaabbb, \dots\}$$

$$G = (V, T, P, S)$$

$$V = \{S\}$$

$$T = \{a, b\}$$

$$P = \{S \rightarrow a | asb\}$$

S is start symbol

4) obtain a grammar to generate the following language

$$L = \{a^n b^{n+1} : n \geq 0\}$$

$$G = (V, T, P, S)$$

$$V = \{S\}$$

$$T = \{a, b\}$$

$$P = \{S \rightarrow b \mid a S b\}$$

S is start symbol.

5) obtain a grammar to generate the following

$$L = \{a^n b^{n+2} : n \geq 0\}$$

Two extra b 's should be generated so the final grammar to generate given lang is correct

$$G = (V, T, P, S)$$

$$V = \{S\}$$

$$T = \{a, b\}$$

$$P = \{S \rightarrow bb \mid a S b\}$$

S is start symbol

6) obtain a grammar to generate the following language $L = \{a^n b^{2n} : n \geq 0\}$

$$G = (V, T, P, S)$$

$$V = \{S\}$$

$$T = \{a, b\}$$

$$P = \{S \rightarrow \epsilon \mid a S b b\}$$

S is start symbol.

- (17) * obtain a grammar G & generating set of all palindrome over $\Sigma = \{a, b\}^*$

$$G = (V, T, P, S)$$

$$V = \{S\}$$

$$T = \{a, b\}$$

$$P = \left\{ \begin{array}{l} S \rightarrow \epsilon \\ S \rightarrow a|b \\ S \rightarrow asa|bsb \end{array} \right\}$$

S is start symbol.

- (18) obtain a grammar to generate the following

$$L = \{ww^R \text{ where } w \in \{a, b\}^*\}$$

$$L = \{aa, bb, abba, baab, \dots\}$$

$$G = (V, T, P, S)$$

$$V = \{S\}$$

$$T = \{a, b\}$$

$$P = \{S \rightarrow \epsilon \mid asa \mid bsb\}$$

S is start symbol

- (19) obtain a grammar to generate a language consisting of all non-palindrome over $\{a, b\}^*$

$$G = (V, T, P, S)$$

$$V = \{S, A, B\}$$

$$T = \{a, b\}$$

$$P = \left\{ \begin{array}{l} S \rightarrow asa \mid bsb \\ S \rightarrow A \\ A \rightarrow aBb \mid bBa \\ B \rightarrow aB \mid bB \mid \epsilon \end{array} \right\}$$

S is start symbol

20) obtain grammar to generate the language ^⑤
 $L = \{0^m 1^m 2^n \mid m \geq 1 \text{ \& } n \geq 0\}$

$$L = \{ \underbrace{0^m}_A \underbrace{1^m}_B 2^n \mid m \geq 1 \text{ \& } n \geq 0 \}$$

$$S \rightarrow A B$$

The variable A should produce more no. of 0's and followed by m no. of 1's

B should produce any no. of 2's

$$B \rightarrow \epsilon / 2B$$

$$S \rightarrow AB$$

$$A \rightarrow 01 / 0A1$$

$$B \rightarrow \epsilon / 2B$$

$$G = (V, T, P, S)$$

$$V = \{S, A, B\}$$

$$T = \{0, 1, 2\}$$

$$P = \{ S \rightarrow AB, \\ A \rightarrow 01 / 0A1 \\ B \rightarrow \epsilon / 2B \}$$

S is start symbol

21) obtain the grammar to generate the language $L = \{w \mid n_a(w) = n_b(w)\}$

ex: abba, baab

$$G = (V, T, P, S)$$

$$V = \{S\}$$

$$T = \{a, b\}$$

$$P = \{ S \rightarrow \epsilon / asb / bsa / ss \}$$

ϵ is start symbol.

Q22 obtain a grammar to generate a string balanced parenthesis.

$$G = (V, T, P, S)$$

$$V = \{S\}$$

$$T = \{ (,) \}$$

$$P = \{ S \rightarrow (S) \mid SS \mid \epsilon \}$$

S is start symbol

Q23 obtain a grammar to generate the language

$$L = \{ a^i b^j \mid i \neq j, i \geq 0 \text{ \& } j \geq 0 \}$$

$$L = \{ a^i b^j \mid i \neq j, i \geq 0, j \geq 0 \}$$

$$G = (V, T, P, S)$$

$$V = \{S, A, B\}$$

$$T = \{a, b\}$$

$$P = \{ S \rightarrow aSb \mid A \mid B$$

$$A \rightarrow a \mid aA$$

$$B \rightarrow b \mid bB$$

S is start symbol

$$L = \{ 0^i 1^j \mid i \neq j, i \geq 0, j \geq 0 \}$$

$$S \rightarrow 0S1 \mid A \mid B$$

$$A \rightarrow 0 \mid 0A$$

$$B \rightarrow 1 \mid 1B$$

$$G = (V, T, P, S)$$

$$V = \{S, A, B\}$$

$$T = \{0, 1\}$$

$$P = \{ S \rightarrow 0S1 \mid A \mid B$$

$$A \rightarrow 0 \mid 0A$$

$$B \rightarrow 1 \mid 1B \}$$

S is start symbol.

Q24 obtain a grammar to generate the language

$$L = \{ a^n b^m \mid n \geq 0, m > n \}$$

$$\begin{matrix} n=0 & n=1 & n=2 \\ m \geq 1 & m \geq 2 & m \geq 3 \end{matrix}$$

$$L = \{ \epsilon b b^*, \underline{a} b b b^*, \underline{a a} b b b b^* - \dots \}$$

$$G = (V, T, P, S)$$

$$V = \{S, B\}$$

$$T = \{a, b\}$$

$$P = \{S \rightarrow asb|B, \quad [\text{generating } a^n b^n | n \geq 0]$$

$$B \rightarrow bB|b \} \quad [\text{generate one or more b's}]$$

S is start symbol

35) obtain a grammar to generate the language

$$L = \{a^n b^{n-3} | n \geq 3\}$$

$$L = \{ \overset{n=3}{aaa}, \overset{n=4}{aaaa}b, \overset{n=5}{aaaaa}bb, \dots \}$$

$$G = (V, T, P, S)$$

$$V = \{S, A\}$$

$$T = \{a, b\}$$

$$P = \{S \rightarrow aaaA, \\ A \rightarrow aAb| \epsilon\}$$

S is start symbol.

Derivation:

"The process of obtaining string of terminal and/or non terminal from the start symbol by applying some or all production is called derivation."

$\Rightarrow$ If ~~only~~ ~~or~~ ~~more~~, only one production are applied to get string is called one step derivation. one or more production are applied to get string $\alpha\beta$ from A then

$$\underline{A \Rightarrow^+ \alpha\beta}$$

If zero or more production are applied to get the string $\alpha\beta\gamma$ from A then we write

$$\underline{\underline{A \Rightarrow^* \alpha\beta\gamma}}$$

(i) Leftmost derivation:

In the derivation process, if a left most variable is replaced at every step then the derivation is said to be leftmost.

(ii) Rightmost derivation:

In the derivation process, if a right most variable is replaced at every step then the derivation is said to be rightmost.

1) consider a grammar shown below from which any arithmetic expression can be obtained.
 $E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid \text{id}$
 E is start symbol. obtain the string $\text{id} + \text{id} * \text{id}$ by applying leftmost & rightmost derivation.

Leftmost

$$\begin{aligned} E &\xrightarrow{\text{lm}} E + E \\ &\Rightarrow \text{id} + E \\ &\Rightarrow \text{id} + E * E \\ &\Rightarrow \text{id} + \text{id} * E \\ &\Rightarrow \text{id} + \text{id} * \text{id} \end{aligned}$$

The string $\text{id} + \text{id} * \text{id}$ is obtained from the start symbol E by applying leftmost derivation & can be written as

$$\underline{\underline{E \xrightarrow{\text{lm}} \text{id} + \text{id} * \text{id}}}$$

Rightmost

$$\begin{aligned} E &\Rightarrow E + E \\ &\Rightarrow E + E * \text{id} \\ &\Rightarrow E + E * \text{id} \end{aligned}$$

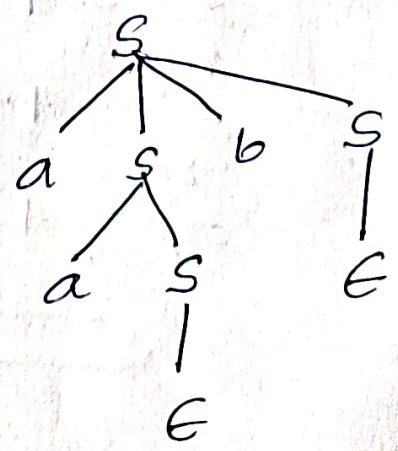
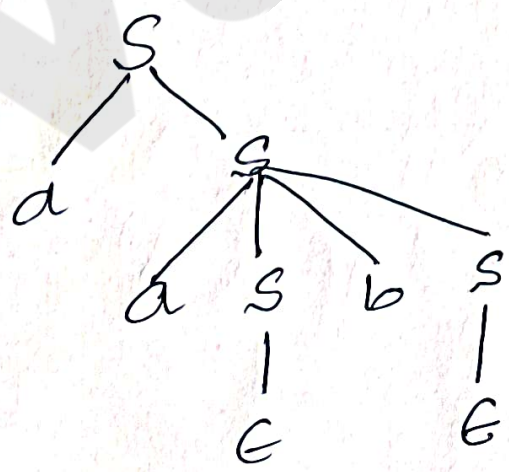
$$\therefore E \xRightarrow{rm} \underline{\underline{Ed + Ed * Ed}}$$

Q1) obtain the leftmost derivation for the string aaababbba using following grammar

$$\begin{aligned} S &\rightarrow aB/bA \\ A &\rightarrow aS/bAA/a \\ B &\rightarrow bS/aBB/b \end{aligned}$$

$$\begin{aligned} S &\xRightarrow{lm} aB && (\text{Applying } S \rightarrow aB) \\ &\Rightarrow aaBB && (B \rightarrow aBB) \\ &\Rightarrow aaabBB && (B \rightarrow aBB) \\ &= aaabBB && (B \rightarrow b) \\ &= aaabbB && (B \rightarrow b) \\ &= aaabbaBB && (B \rightarrow aBB) \\ &= aaabtabB && (B \rightarrow b) \\ &= aaabtabbs && (B \rightarrow bS) \\ &= aaabtabbA && (S \rightarrow bA) \\ &= \underline{\underline{aaabtabbba}} && (A \rightarrow a) \end{aligned}$$

Q show that grammar below is ambiguous
the grammar is ambiguous because the
sentence aab has two parse trees.



③ consider the grammar $E \rightarrow +EE \mid *EE \mid -EE \mid x \mid y$

(a) find LMD & RMD for string $-+*xyxy$

(b) is the grammar ambiguous? Justify

(a) Leftmost derivation.

$$\begin{aligned}
 E &\xrightarrow{lm} -EE \\
 &\xrightarrow{lm} -+EEE \\
 &\xrightarrow{lm} -+*EEEE \\
 &\xrightarrow{lm} -+*xEEE \\
 &\xrightarrow{lm} -+*xyEE \\
 &\xrightarrow{lm} -+*xyxE \\
 &\xrightarrow{lm} -+*xyxy
 \end{aligned}$$

(b) Rightmost derivation

$$\begin{aligned}
 E &\xrightarrow{rm} -EE \\
 &\xrightarrow{rm} -EY \\
 &\xrightarrow{rm} -+EEY \\
 &\xrightarrow{rm} -+*EEXY \\
 &\xrightarrow{rm} -+*EYXY \\
 &\xrightarrow{rm} -+*xyxy
 \end{aligned}$$

ex ① $S \Rightarrow aB$

$a(aBB)$
 $aaB(aBB)$
 $aaBaB(bS)$
 $aaBaB(bA)$
 $aaBaB(bbA)$
 $aaBa(bbba)$
 $aaabba bbba$
 $aaabba bbba$

$aaabba bbba$

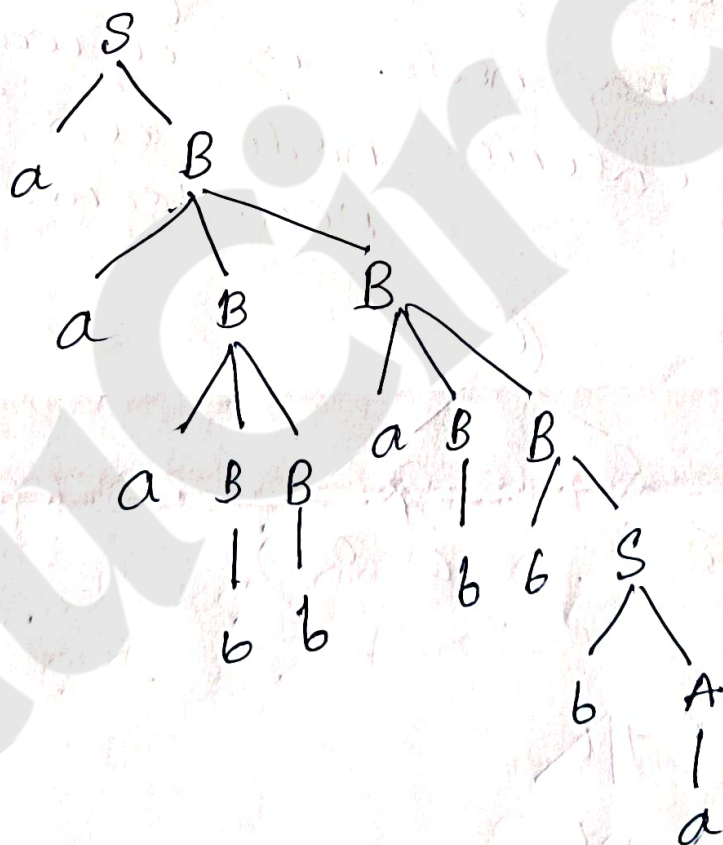
Grammar

$S \rightarrow aB | bA$

$A \rightarrow aS | bAA | a$

$B \rightarrow bS | aBB | b$

rightmost
derivation



parse tree

It is a graphical representation for the derivation of the given production rule for a given string.

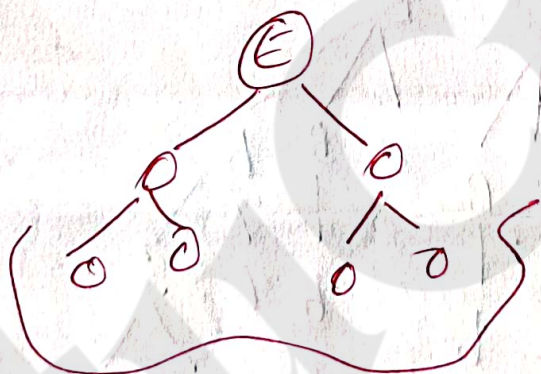
Ex.

* The root node is always a node indicating start symbol

* The der is read from left to right

* The leaf node is always terminal nodes

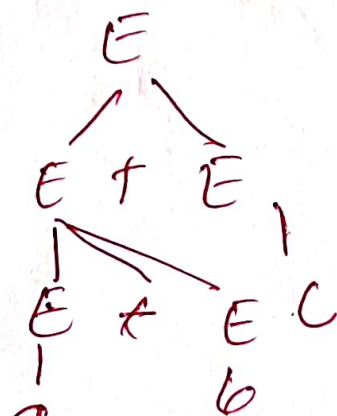
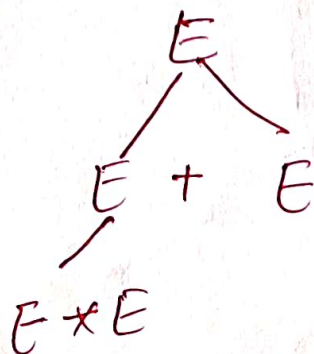
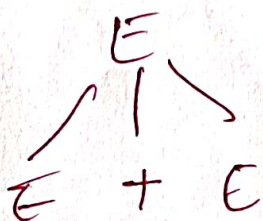
* The interior nodes are always non-terminal



$$E = E + E$$

$$E = E * E$$

$$E = a | b | c$$



Ambiguity [ambiguous grammar]:

A grammar is said to be ambiguous if there exists, one

more than one leftmost derivation (or)
more than one rightmost derivation (or)
more than one parse tree for given input string.

⇒ If the grammar is not ambiguous then we call unambiguous, then we call

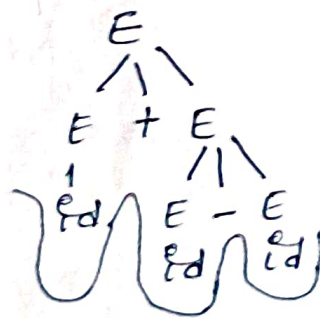
⇒ If the grammar has ambiguity, then it is not good for compiler construction

⇒ no method can automatically detect & remove the ambiguity, but we can remove ambiguity by rewriting the whole grammar without ambiguity.

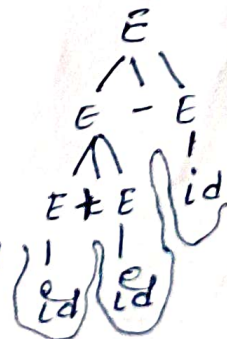
⑨ check whether the grammar is ambiguous or not. the string is id+id-id

$E \rightarrow E + E$
 $E \rightarrow E - E$
 $E \rightarrow id$

$E \Rightarrow E + E$
 $id + E$
 $id + E - E$
 $id + id - E$
 $id + id - id$



$E \Rightarrow E - E$
 $E + E - E$
 $id + E - E$
 $id + id - E$
 $id + id - id$



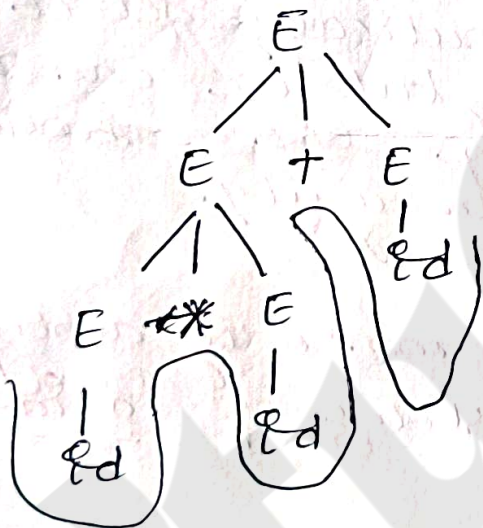
The given grammar is ambiguous.

Q Given grammar, $E \rightarrow E + E$
 $E \rightarrow E * E$
 $E \rightarrow (E)$
 $E \rightarrow id$

obtain derivation tree of expression
 (i) $id * id + id$ (ii) $(id + id) * id$

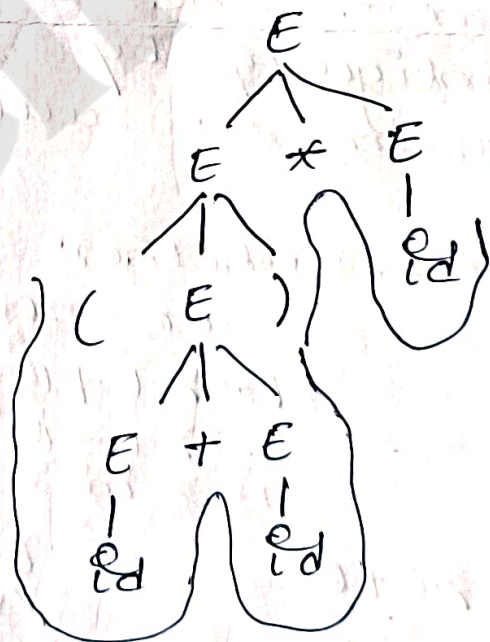
(i) $id * id + id$

$E \Rightarrow E + E$
 $\Rightarrow E * E + E$
 $\Rightarrow id * id + id$



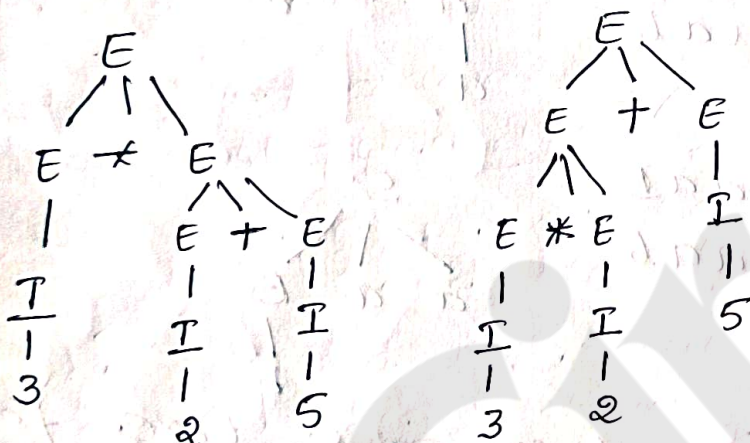
(ii) $(id + id) * id$

$E \Rightarrow E * E$
 $\Rightarrow (E) * E$
 $\Rightarrow (E + E) * E$
 $\Rightarrow (id + id) * id$



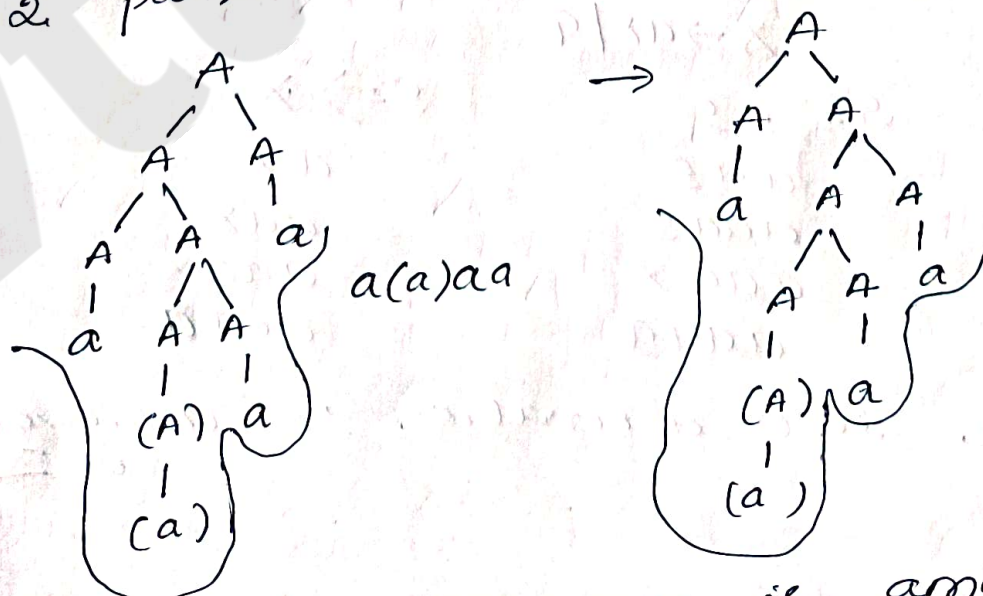
② consider a grammar with production rule check whether grammar is ambiguous or not? string $3 * 2 + 5$.

$E \rightarrow I$
 $E \rightarrow E + E$
 $E \rightarrow E * E$
 $E \rightarrow (E)$
 $I \rightarrow \epsilon | 0 | 1 | 2 | \dots | 9$



Since there are two parse for single string the grammar is ambiguous.

③ check whether the grammar is ambiguous or not. $A \rightarrow (A)$, $A \rightarrow a$, $A \rightarrow AA$. For the string $a(a)aa$. It can generate 2 parse tree

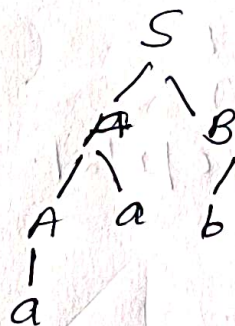


the given grammar is ambiguous.

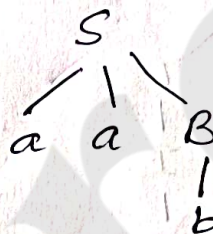
④ $\Rightarrow$ the grammar ambiguous
 $S \rightarrow AB | aAB$ string aab
 $A \rightarrow a | AA$
 $B \rightarrow b$

Leftmost

$S \Rightarrow AB$
 AaB
 aAB
 aab



$S \Rightarrow aAB$
 $\Rightarrow aab$



Since there are two parse tree for the string aab, so given grammar is ambiguous.

⑤ $\Rightarrow$ the following grammar ambiguous
 $S \rightarrow as | x$
 $x \rightarrow ax | a$

$S \Rightarrow as$
 aas
 $aaas$
 $aaax$
 $aaaa$

$S \Rightarrow x$
 ax
 aax
 $aaax$
 $aaaa$

Since two derivation $\therefore$ ambiguous

Q Is the following grammar ambiguous?

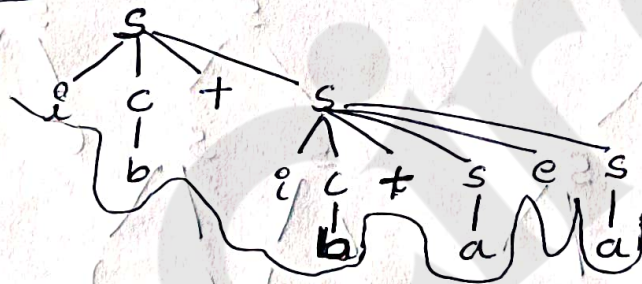
$$S \rightarrow ic + s \mid ic + ses \mid a$$

$$c \rightarrow b$$

The string $ib + ib + aea$ can be obtained by applying the leftmost derivation as shown below

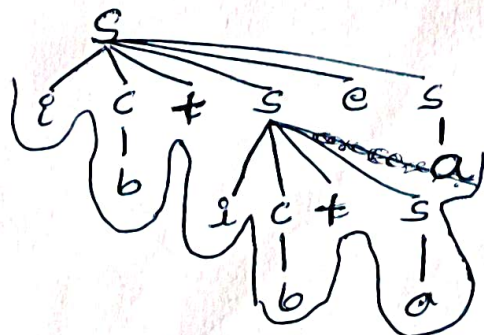
$$\begin{aligned} S &\Rightarrow ic + s \\ &\Rightarrow ib + s \\ &\Rightarrow ib + ic + ses \\ &\Rightarrow ib + ib + ses \\ &\Rightarrow ib + ib + aea \end{aligned}$$

parse tree



$$\begin{aligned} S &\Rightarrow ic + ses \\ &\Rightarrow ib + ses \\ &\Rightarrow ib + ic + ses \\ &\Rightarrow ib + ib + ses \\ &\Rightarrow ib + ib + aea \end{aligned}$$

parse tree

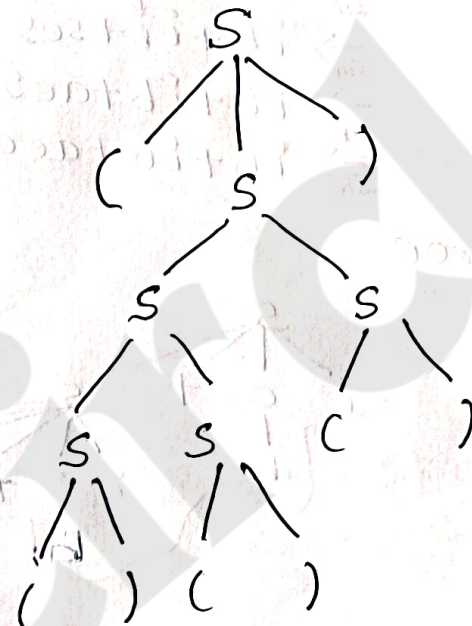
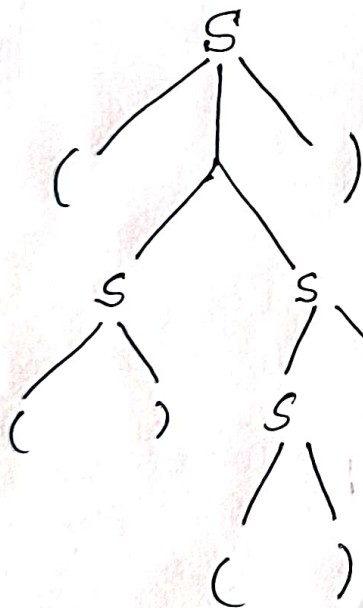


∴ Since it has two parse tree for same string, so grammar is ambiguous.

⑦ show that the grammar given below to generate non empty balanced parentheses is ambiguous.

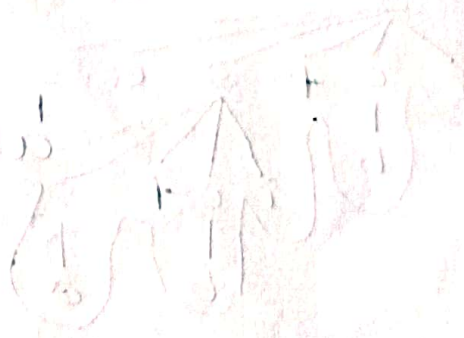
$$S \rightarrow SS | (S) | ()$$

The sentence $()()()$ has two parse tree as given below.



It has more than one derivation tree is

$\underline{\underline{()()()}}$



PUSHDOWN Automata

A pushdown automata is way to implement a context free grammar in a similar way we design DFA a Regular Grammar.

- ⇒ A 'DFA' can remember a "finite" amount of information but,
- ⇒ A PDA can remember a "infinite" amount of information.

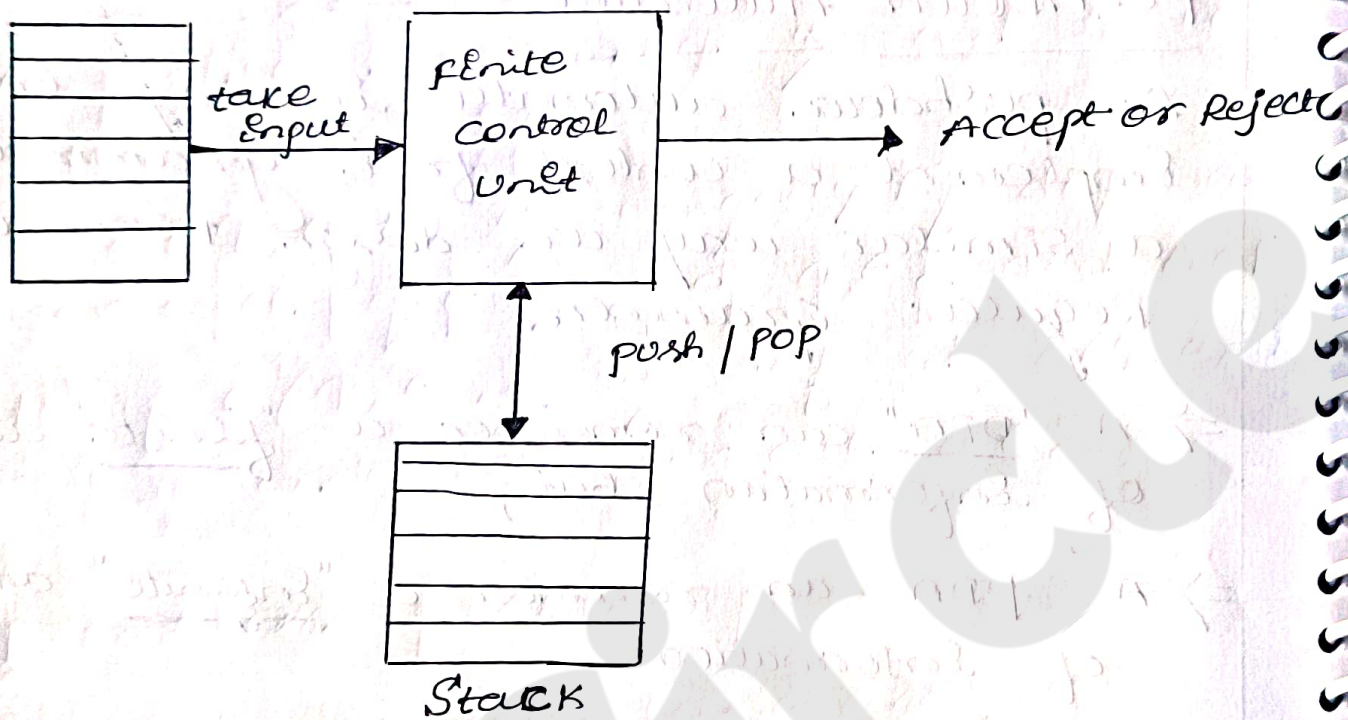
Basically a PDA is

"finite state machine" + "A stack"

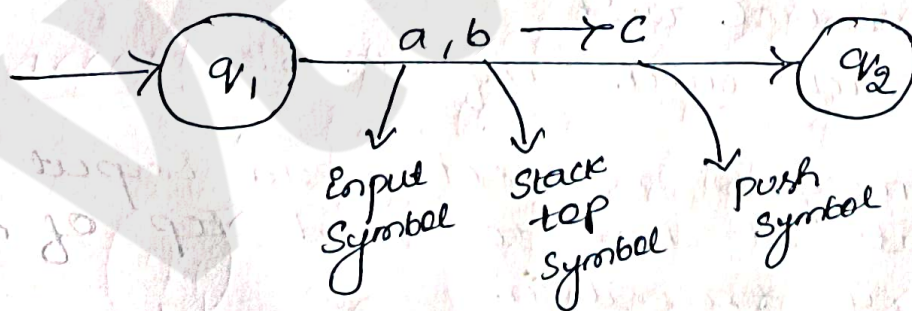
⇒ A PDA has 3 component

- * An input tape
- * A control unit
- * A stack with infinite size.

⇒ A PDA may or may not read input symbol but it has to read top of stack in every transition.



The following figure shows a transition in a PDA from a state q_1 to q_2 labeled as $a, b \rightarrow c$.



Define PDA

PDA can be mathematically, a PDA defined as

$$M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$$

- (i) Q is a set of states
- (ii) Σ is the input symbol alphabet
- (iii) Γ stack alphabet
- (iv) δ is the transition function
- (v) $q_0 \in Q$ is the start state
- (vi) $z_0 \in \Gamma$ is initial stack
- (vii) $F \subseteq Q$ is a set of accepting states.

Terminologies:

Instantaneous Description ID (ID):

The ID of PDA is represented by triple (q, w, s) where

$q \rightarrow$ state

$w \rightarrow$ unconsume input

$s \rightarrow$ stack content.

Turnstile notation ($\vdash$)

It is used for connecting pairs of ID's that represent one or many moves of PDA.

The process of transition denoted by turnstile symbol ($\vdash$)

$\vdash$

one move

$\vdash^*$

sequence of moves.

Deterministic PDA (push down Automata)

A pushdown automata is said to be deterministic if

(i) $\delta(q, a, A)$ is uniquely defined for each $q \in Q$ and $a \in \Sigma$ and $A \in \Gamma$

(ii) If $\delta(q, \epsilon, A)$ is defined, $\delta(q, a, A)$ is undefined for all $a \in \Sigma$.

problems:

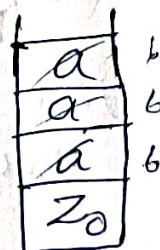
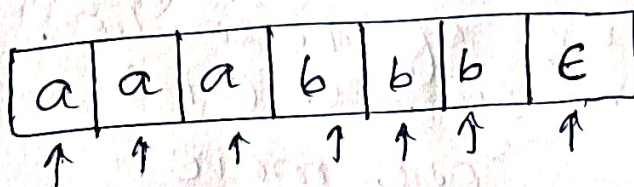
① Design PDA for language,
 $A^n B^n = \{a^n b^n : n \geq 1\}$

sol whenever a comes, push into stack
whenever b comes, pop 'a' from stack.

accept: $\rightarrow$ Input string should be consumed
 $\rightarrow$ should be in final state
 $\rightarrow$ Stack should be empty

$$M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$$

$$L = \{ab, aabb, aaabbb, \dots\}$$



$$\delta(q_0, a, z_0) = (q_0, az_0)$$

$$\delta(q_0, a, a) = (q_0, aaz_0)$$

$$\delta(q_0, a, a) = (q_0, aaz_0) \times$$

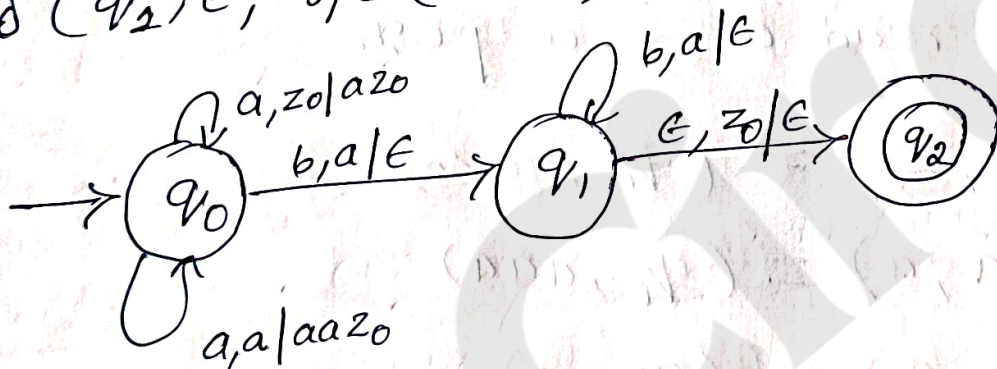
$$\delta(q_0, b, a) = (q_1, \epsilon)$$

$$\delta(q_0, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, z_0) = (q_2, \epsilon)$$



string aabb

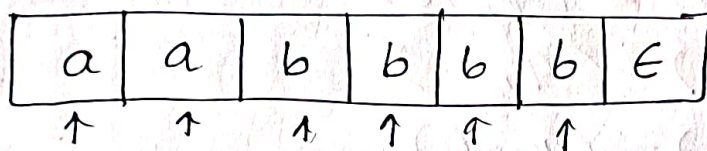
$$\begin{aligned}
 (q_0, aabb, z_0) &\vdash (q_0, abb, az_0) \\
 &\vdash (q_0, bb, aaz_0) \\
 &\vdash (q_0, b, az_0) \\
 &\vdash (q_1, \epsilon, z_0) \\
 &\vdash (q_2, \epsilon)
 \end{aligned}$$

$$M = \{ \{q_0, q_1, q_2\}, \{a, b\}, \{z_0 a\}, \delta, q_0, z_0, q_2 \}$$

2) construct PDA $L = \{a^n b^{2n} \mid n \geq 1\}$

$$M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$$

$$L = \{abb, aabbbb, \dots\}$$



logic 1: read a $\rightarrow$ 2as
 read b $\rightarrow$ Pop

logic 2: read a $\rightarrow$ a push

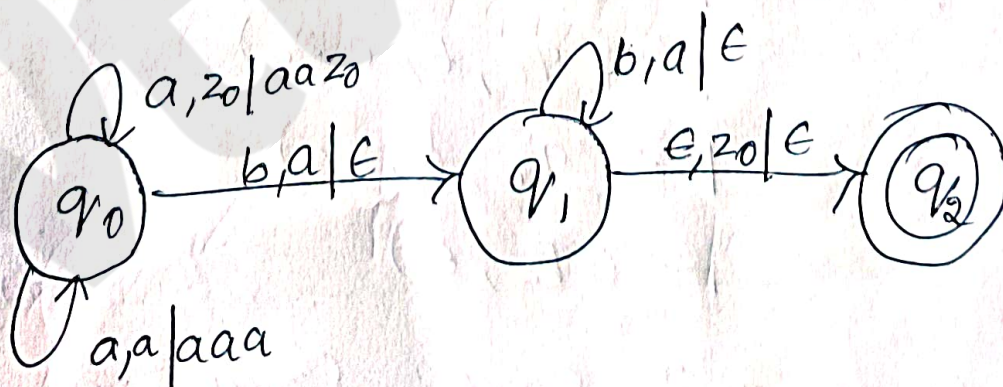
$$\delta(q_0, a, z_0) = (q_0, aa z_0)$$

$$\delta(q_0, a, a) = (q_0, aaa)$$

$$\delta(q_0, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, z_0) = (q_2, \epsilon)$$



$$M = (\{q_0, q_1, q_2\}, \{a, b\}, \{z_0, a\}, \delta, q_0, z_0, \{q_2\})$$

③ PDA construction PDA lang $L = \{wcw^R \mid w \in (a+b)^*\}$

$M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$

$L = \{aba\bar{c}aba, abab\bar{c}baba, \dots\}$

a
b
a
z_0

$\delta(q_0, a, z_0) = (q_0, az_0)$

$\delta(q_0, b, z_0) = (q_0, bz_0)$

$\delta(q_0, a, a) = (q_0, aa)$

$\delta(q_0, a, b) = (q_0, ab)$

$\delta(q_0, b, a) = (q_0, ba)$

$\delta(q_0, b, b) = (q_0, bb)$

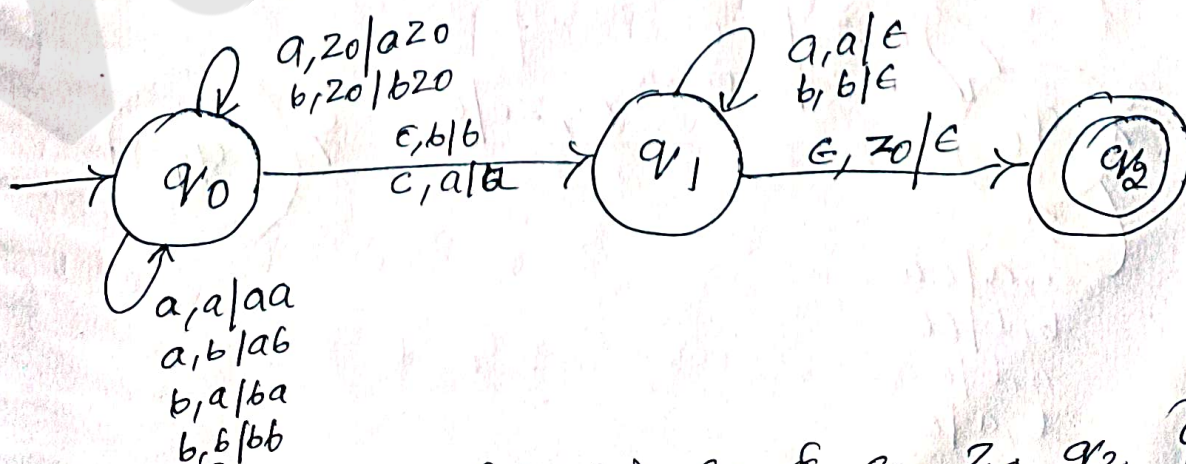
$\delta(q_0, c, a) = (q_1, a)$

$\delta(q_0, c, b) = (q_1, b)$

$\delta(q_1, a, a) = (q_1, \epsilon)$

$\delta(q_1, b, b) = (q_1, \epsilon)$

$\delta(q_1, \epsilon, z_0) = (q_2, \epsilon)$



$M = \{q_0, q_1, q_2\}, \{a, b\}, \{a, b\}, \delta, q_0, z_0, q_2\}$

④ construct PDA for $L = \{w \mid n_a(w) = n_b(w)\}$

$L = \{\epsilon, ab, aabb, abab, abba, bbaa, \dots\}$

1. stack empty, a, b push

2. top $\rightarrow a$ input a — push

3. top $\rightarrow a$ input b — pop

3. top $\rightarrow b$ input b — push
input a — pop

$$\delta(q_0, a, z_0) = (q_0, az_0)$$

$$\delta(q_0, b, z_0) = (q_0, bz_0)$$

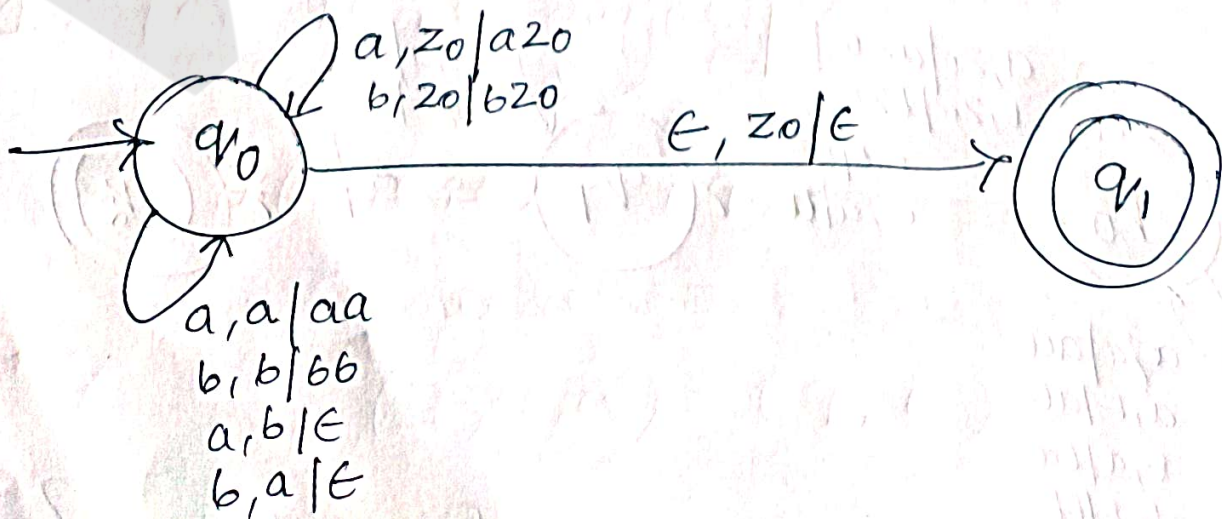
$$\delta(q_0, a, a) = (q_0, aa)$$

$$\delta(q_0, b, b) = (q_0, bb)$$

$$\delta(q_0, a, b) = (q_0, \epsilon)$$

$$\delta(q_0, b, a) = (q_0, \epsilon)$$

$$\delta(q_0, \epsilon, z_0) = (q_1, \epsilon)$$



$PDA \rightarrow M = (\{q_0, q_1\}, \{a, b\}, \{a, b, z_0\}, \delta, \{q_0\}, z_0, \{q_2\})$

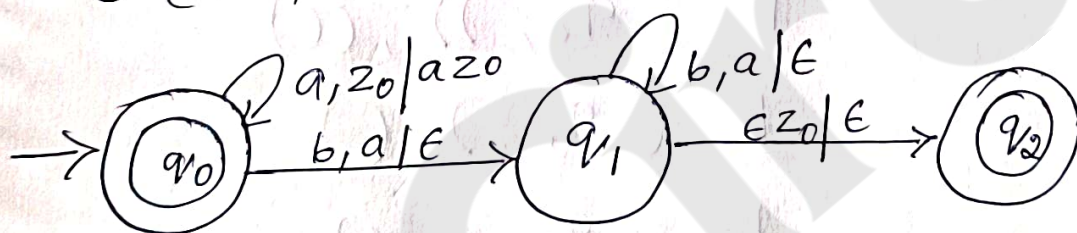
⑤ construct PDA $A^n B^n = \{a^n b^n \mid n \geq 0\}$
 $L = \{\epsilon, aabb, aaabbb, \dots\}$

$$\delta(q_0, a, z_0) = (q_0, a z_0)$$

$$\delta(q_0, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, z_0) = (q_2, \epsilon)$$



$$(q_0, aabb, z_0) \vdash (q_0, abb, a z_0)$$

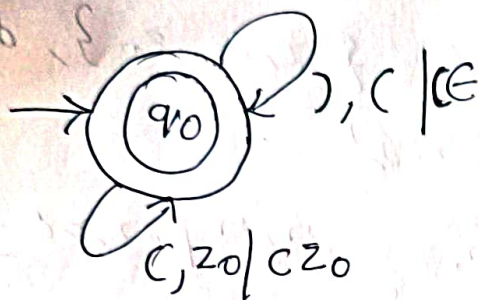
$$\vdash (q_0, bb, aa)$$

$$\vdash (q_1, b, a)$$

$$\vdash (q_1, \epsilon, \epsilon)$$

⑥ Design PDA for balanced parentheses
 $\{()^n \mid n \geq 0\}$ $\{w \in \{ , \}^*\}$

logic: (push into stack
) pop from stack.



$$\delta(q_0, c, z_0) = (q_0, cz_0)$$

$$\delta(q_0,), (c) = (q_0, \epsilon)$$

$$M = \{q_0, \{c, \epsilon\}, \delta, z_0, \Gamma, F\}$$

$$\delta(q_0, (c), z_0) \vdash (q_0, (\underline{c}), c)$$

$$\vdash (q_0,), (c)$$

$$\vdash (q_0,), (c)$$

$$\vdash (q_0, \epsilon, \epsilon)$$

Equivalence of PDA :

construct CFG to PDA

1. for Non terminal / variable

$$A \rightarrow \alpha$$

$$\delta(q, \epsilon, A) = (q, \alpha)$$

2. for terminal

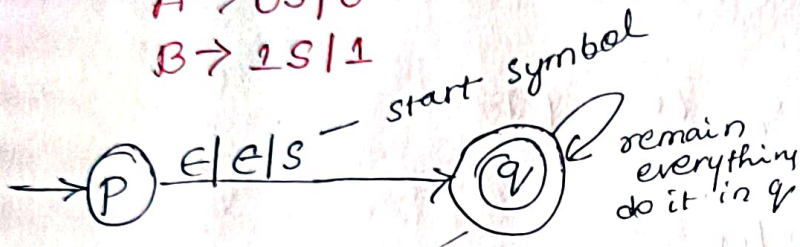
$$\delta(q, a, a) = (q, \epsilon)$$

① construct PDA for grammar:

$$S \rightarrow AB$$

$$A \rightarrow 0S \mid 0$$

$$B \rightarrow 1S \mid 1$$



$$\delta(q_0, \epsilon, S) = (q_1, AB)$$

$$\delta(q_1, \epsilon, A) = (q_1, 0S)$$

$$\delta(q_1, \epsilon, A) = (q_1, 0)$$

$$\delta(q_1, \epsilon, B) = (q_1, 1S)$$

$$\delta(q_1, \epsilon, B) = (q_1, 1)$$

$$\delta(q_1, 0, 0) = (q_1, \epsilon)$$

$$\delta(q_1, 1, 1) = (q_1, \epsilon)$$

② Construct PDA for grammar

$$E \rightarrow E+T \quad E \rightarrow T \quad T \rightarrow T * F \quad T \rightarrow F \quad F \rightarrow (E), F \rightarrow id$$



$$\delta(q_0, \epsilon, \epsilon) = (q_1, E+T)$$

$$\delta(q_1, \epsilon, \epsilon) = (q_1, T)$$

$$\delta(q_1, \epsilon, T) = (q_1, T * F)$$

$$\delta(q_1, \epsilon, T) = (q_1, F)$$

$$\delta(q_1, \epsilon, F) = (q_1, (E))$$

$$\delta(q_1, \epsilon, F) = (q_1, id)$$

$$\delta(q_1, id, id) = (q_1, \epsilon)$$

$$\delta(q_1, (, () = (q_1, \epsilon)$$

$$\delta(q_1,),) = (q_1, \epsilon)$$

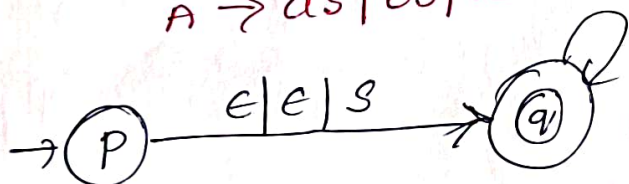
$$\delta(q, +, +) = (q, \epsilon)$$

$$\delta(q, *, *) = (q, *)$$

③ construct CFG to PDA.

$$S \rightarrow aAA$$

$$A \rightarrow as | bs | a$$



$$\delta(q, \epsilon, s) = (q, aAA)$$

$$\delta(q, \epsilon, A) = (q, as)$$

$$\delta(q, \epsilon, A) = (q, bs)$$

$$\delta(q, \epsilon, A) = (q, a)$$

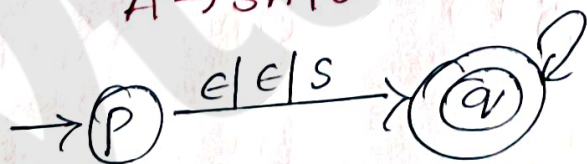
$$\delta(q, a, a) = (q, \epsilon)$$

$$\delta(q, b, b) = (q, \epsilon)$$

④ construct CFG to PDA

$$S \rightarrow AA | a$$

$$A \rightarrow SA | b$$



$$\delta(q, \epsilon, s) = (q, AA)$$

$$\delta(q, \epsilon, s) = (q, a)$$

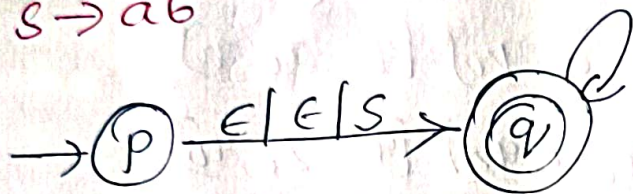
$$\delta(q, \epsilon, A) = (q, SA)$$

$$\delta(q, \epsilon, A) = (q, b)$$

$$\delta(q, a, a) = (q, \epsilon)$$

$$\delta(q, b, b) = (q, \epsilon)$$

⑤ $s \rightarrow a S b$ convert into PDA.
 $s \rightarrow ab$

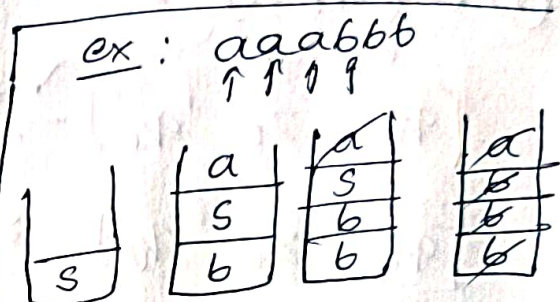


$$\delta(q, \epsilon, s) = (q, asb)$$

$$\delta(q, \epsilon, s) = (q, ab)$$

$$\delta(q, a, a) = (q, \epsilon)$$

$$\delta(q, b, b) = (q, \epsilon)$$



Construction from PDA to CFG

① Design PDA to CFG,

$$M = (\{q_0, q_1\}, \{a, b\}, \{a, z_0\}, \delta, q_0, z_0, F)$$

$$\delta(q_0, a, z_0) = (q_0, a z_0)$$

$$\delta(q_0, a, a) = (q_0, aa)$$

$$\delta(q_0, b, a) = (q_1, a)$$

$$\delta(q_1, b, a) = (q_1, a)$$

$$\delta(q_1, a, a) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, z_0) = (q_1, \epsilon)$$

Solution

$$CFG(\Sigma) = (V, T, P, S)$$

$$V = \{S, T\}$$

$$\therefore V = (S, [q_0, a, q_0] [q_0, a, q_1] [q_1, a, q_0] [q_1, a, q_1] [q_0, z_0 q_0] [q_0 z_0 q_1] [q_1, z_0 q_0] [q_1, z_0 q_1])$$

$$T = \{a, b\}$$

$$S \rightarrow [q_0, z_0, q_0]$$

$$S \rightarrow [q_0, z_0, q_1]$$

$$\begin{aligned}
 \textcircled{1} \quad \delta(q_0, a, z_0) &= (q_0, a, z_0) \\
 [q_0, z_0, q_0] &\rightarrow a [q_0, a, q_0] [q_0, z_0, q_0] \\
 [q_0, z_0, q_0] &\rightarrow a [q_0, a, q_1] [q_1, z_0, q_0] \\
 [q_0, z_0, q_1] &\rightarrow a [q_0, a, q_0] [q_0, z_0, q_1] \\
 [q_0, z_0, q_1] &\rightarrow a [q_0, a, q_1] [q_1, z_0, q_1] \\
 [q_0, z_0, q_1] &\rightarrow a [q_0, a, q_1] [q_1, z_0, q_1] \quad \text{same}
 \end{aligned}$$

$$\begin{aligned}
 \textcircled{2} \quad \delta(q_0, a, a) &= (q_0, a, a) \\
 [q_0, a, q_0] &\rightarrow a [q_0, a, q_0] [q_0, a, q_0] \\
 [q_0, a, q_0] &\rightarrow a [q_0, a, q_1] [q_1, a, q_0] \\
 [q_0, a, q_0] &\rightarrow a [q_0, a, q_0] [q_0, a, q_1] \\
 [q_0, a, q_0] &\rightarrow a [q_0, a, q_0] [q_0, a, q_1] \\
 [q_0, a, q_1] &\rightarrow a [q_0, a, q_1] [q_1, a, q_1]
 \end{aligned}$$

$$\begin{aligned}
 \textcircled{3} \quad \delta(q_0, b, a) &= (q_1, a) \\
 [q_0, a, q_0] &\rightarrow b [q_1, a, q_0] \\
 [q_0, a, q_1] &\rightarrow b [q_1, a, q_1]
 \end{aligned}$$

$$\begin{aligned}
 \textcircled{4} \quad \delta(q_1, b, a) &= (q_1, a) \\
 [q_1, a, q_0] &\rightarrow b [q_1, a, q_0] \\
 [q_1, a, q_1] &\rightarrow b [q_1, a, q_1]
 \end{aligned}$$

$$\textcircled{5} \delta(q_1, a, a) = (q_1, \epsilon)$$

$$[q_1, a, q_1] \rightarrow a$$

$$\textcircled{6} \delta(q_1, \epsilon, z_0) = (q_1, \epsilon)$$

$$[q_1, z_0, q_1] \rightarrow \epsilon //$$

$\textcircled{2}$ construct CFG from given PDA

$$M = (\{q, p\}, \{0, 1\}, \{x, z\}, \delta, q_0, z, F)$$

$$\begin{aligned} \delta(q, 0, x) &= (p, x) \\ \delta(q, 1, z) &= (q, xz) \\ \delta(q, 1, x) &= (q, xx) \\ \delta(q, \epsilon, x) &= (q, \epsilon) \end{aligned}$$

$$\begin{aligned} \delta(p, 1, x) &= (p, \epsilon) \\ \delta(p, 0, z) &= (q, z) \end{aligned}$$

sol CFG $(G) = (V, T, P, S)$

$$V = (S, [q_0, x, p], [q, x, q], [p, x, p], [p, x, q], [q, z, p], [q, z, q], [p, z, p], [p, z, q])$$

$$T = \{0, 1\}$$

$$S \rightarrow [q_0, z, q]$$

$$S \rightarrow [q_0, z, p]$$

$$\textcircled{1} \delta(q, 1, z) = (q, x, z)$$

$$[q, \underline{1}, q] \rightarrow 1 [q, x, q] [q, z, q]$$

$$[q, 1, q] \rightarrow 1 [q, x, p] [p, z, q]$$

$$[q, 1, p] \rightarrow 1 [q, x, q] [q, z, p]$$

$$[q, z, p] \rightarrow 1 [q, x, p] [p, z, p]$$

$$\textcircled{2} \delta(q, 1, x) = (q, x, x)$$

$$[q, \underline{1}, q] \rightarrow x [q, x, q] [q, x, q]$$

$$[q, 1, q] \rightarrow x [q, x, p] [p, x, q]$$

$$[q, x, p] \rightarrow x [q, x, q] [q, x, p]$$

$$[q, x, p] \rightarrow x [q, x, p] [p, x, p]$$

$$\textcircled{3} \delta(q, \epsilon, x) = (q, \epsilon)$$

$$[q, \underline{\epsilon}, q] \rightarrow \epsilon$$

$$\textcircled{4} \delta(q, 0, x) = (p, x)$$

$$[q, \underline{0}, q] \rightarrow 0 [p, x, q]$$

$$[q, 0, p] \rightarrow 0 [p, x, p]$$

$$\textcircled{5} \delta(p, 0, z) = (q, z)$$

$$[p, \underline{0}, q] \rightarrow 0 [q, z, q]$$

$$[p, 0, p] \rightarrow 0 [q, z, p]$$

$$(6) \delta(P, \frac{1}{x}, x) = (P, \epsilon)$$

$$[P, x, P] \rightarrow 1$$

//

$P = \{ S \rightarrow Aa | aB | b | Ca$

$B \rightarrow aB | b$

$C \rightarrow Db | ab | d$

$D \rightarrow d | ab$

$E \rightarrow ab \}$

S is start symbol.